

Samenvatting Object Oriented Programming 2 (C++)

Examen-gerichte samenvatting, gebaseerd op de cursuspagina's van programming-in-cpp.netlify.app.

Inhoud

1. Intro + Hello World
2. Compileren & linken (compilatiepipeline)
3. Makefiles
4. Fundamentals (types, operators, I/O, control flow)
5. Arrays, enums, functies
6. Object Oriented Thinking: abstractie, objecten, klassen, relaties
7. OOP in C++: classes, headers/cpp, namespaces, access modifiers, const methods
8. Constructors + constructor initialization lists
9. Composition
10. Inheritance + overriding
11. Polymorfisme (`virtual`) + virtual destructors
12. Abstract classes & "interfaces" (pure virtual)
13. Pointers + references
14. Dynamic memory (stack vs heap, `new/delete`, destructors)
15. Exceptions
16. File streams (`<fstream>`)
17. Operator overloading
18. Standard library: `std::vector`
19. Snelle examen-checklist

1) Intro + Hello World

1.1 Basisregels

- Een C++ programma start in `int main()`.
- Elke statement eindigt met `;`.
- `#include` wordt door de preprocessor verwerkt.
- `std` is een namespace (bv. `std::cout`).
- `return 0;` betekent "alles ok" (exit status naar OS).

1.2 Minimal voorbeeld

```
#include <iostream>

int main() {
    std::cout << "Hello World" << std::endl;
    return 0;
}
```

2) Compileren & linken: de compilatiepipeline

2.1 Vier stappen

1. **Preprocessing**: verwerkt `#include`, `#define`, ... (tekstuele transformatie).
2. **Compilation**: C++ broncode → assembly.
3. **Assembling**: assembly → object file (`.o`).
4. **Linking**: object files + libraries → executable.

2.2 `gcc` vs `g++`

- `g++` is de C++ compiler-driver en linkt standaard de C++ standaardbibliotheken.

2.3 Separate compilation

- Typisch: `.h` (declaraties/interface) en `.cpp` (definities/implementatie).
- Je compileert per `.cpp` naar `.o` en linkt op het einde alles samen.
- Dubbele definities geven linker errors (One Definition Rule idee).

Voorbeeld (conceptueel):

```
g++ -Wall -o app main.cpp
```

3) Makefiles

3.1 Wat is een makefile?

- Beschrijft targets, hun dependencies en de commando's om te bouwen.
- Vorm (commando-regels starten met een **TAB**):

```
target: deps
    command
```

3.2 Typische targets

- `all`: bouwt alles.
- `clean`: verwijdert object files/binaries.

3.3 Variabelen

```
CC=g++
CFLAGS=-Wall

all:
    $(CC) $(CFLAGS) -o app main.cpp

clean:
    rm -f app *.o
```

4) Fundamentals

4.1 Variabelen & datatypes

- C++ is **statisch/strongly typed**: types liggen vast bij compile-time.
- **Declaratie** vs **definitie**:
 - definitie reserveert opslag
 - **extern** kan iets declareren dat elders gedefinieerd is
- Initialiseer altijd: on-geïnitieerde variabelen bevatten onvoorspelbare waarden.
- Gebruik **const** voor constants i.p.v. **#define**.
- Typegroottes kunnen platform-afhankelijk zijn; voor vaste breedte bestaan fixed-width integer types (zoals in `<stdint.h>`).

Voorbeeld:

```
const int MAX_USERS = 10;
int count = 0;
```

4.2 Operators (high yield)

- Integerdeling: $5/2 \rightarrow 2$.
- Prefix/postfix: **++i** vs **i++**.
- Short-circuit: **&&** en **||** evalueren rechts niet altijd.
- Ternary: **cond ? a : b**.
- Bitwise: **&** **|** **^** **~** **<<** **>>**.
- **sizeof(expr)** geeft bytes.

4.3 I/O met streams

- Output: **std::cout**, fouten: **std::cerr**.
- Input: **std::cin**.
- **<<** insertion, **>>** extraction.
- **std::endl** = newline + flush.
- Manipulators (idee): **std::hex**, **std::dec**, **std::setw**, **std::setfill**.

4.4 Control flow

- `if / else if / else`.
- `switch`:
 - `break` voorkomt fall-through.
 - bij declaraties binnen een `case` vaak braces `{}` gebruiken.

4.5 Loops

- `while`, `do-while`, `for`.
- `break`, `continue`.

5) Arrays, enums en functies

5.1 Arrays

- Vast formaat, index start bij 0.
- Initialisatie:

```
int a[3] = {1, 2, 3};  
int b[10] = {0};
```

- `sizeof(a)/sizeof(a[0])` werkt enkel waar `a` echt een array is.
- Range-based for:

```
for (auto &v : a) {  
    // ...  
}
```

5.2 Enums

- Klassiek:

```
enum Color { Red, Green, Blue };
```

- Scoped (`enum class`) vermijdt name pollution:

```
enum class Color2 { Red, Green, Blue };
```

- Flags: kies machten van 2 en combineer met bitwise OR.

5.3 Functies

- Prototype (declaratie) vs definitie.
 - Default args bestaan.
 - Passing styles:
 - by value (kopie)
 - by pointer
 - by reference (alias)
 - Arrays "decay" naar pointer in parameters → lengte expliciet meegeven.
-

6) Object Oriented Thinking (OOT)

6.1 Abstractie

- Verberg irrelevante details; bied een duidelijke interface.
- Je programmeert tegen *wat* iets doet, niet *hoe*.

6.2 Objecten

- Object = **state (attributes)** + **behavior (methods)**.
- Interactie tussen objecten = messages (method calls).

6.3 Klassen

- Class = blueprint/type.
- Objecten = instances van die class.

6.4 Interface vs implementatie

- Interface: publieke methods.
- Implementatie: private data + interne code.

6.5 Relaties

- **Association**: "gebruikt"; geen ownership.
- **Composition**: parent owns child; lifecycle afhankelijk (child kan niet zinvol zonder parent).
- **Aggregation**: wel een relatie/ownership, maar child kan onafhankelijk bestaan.
- DRY: hergebruik = niet kopiëren/plakken.

6.6 OOP kenmerken (cursus)

- Encapsulation, message passing, composition (objecten uit objecten), types/classes, en substitutability/polymorfisme.
-

7) OOP in C++: classes bouwen

7.1 Class skeleton

```
class Apple {  
public:  
    Apple();  
    int getSeeds() const;  
  
private:  
    int seeds;  
};
```

7.2 Header/.cpp scheiding

- In `.h`: class + method signatures.
- In `.cpp`: method bodies met `ClassName::method`.

7.3 Include guards / `#pragma once`

- Voorkomt multiple inclusion.

7.4 Namespaces

- Gebruik namespaces om clashes te vermijden.
- Cursus-best practice: **geen** `using namespace ...;` in headers.

7.5 `const` methods

- Markeer getters als `const` zodat ze state niet mogen wijzigen.

8) Constructors

8.1 Doel

- Object in geldige state zetten.

8.2 Default constructor-regel

- De compiler genereert een default constructor **alleen** als je zelf geen constructors definieert.

8.3 Overloading

- Meerdere constructors met verschillende parameterlijsten.

8.4 "Most vexing parse"

- `Point origin;` = object.
- `Point origin();` = functie-declaratie (geen object).

9) Composition

9.1 Waarom vaak beter dan inheritance?

- Flexibeler, minder coupling.
- Inheritance wordt vaak te snel ingezet.

9.2 Constructor order + init-list

- Sub-objects worden geconstrueerd vóór de constructor-body.
- Init-list nodig als een sub-object geen default constructor heeft:

```
Television::Television(double inputVoltage)
: embeddedPower(inputVoltage, 3.3), display(inputVoltage, 12) {
}
```

10) Inheritance

10.1 Concept

- Derived class erft state/behavior van base.
- Moet logisch "is-a" zijn.

10.2 Syntax

```
class Derived : public Base {
};
```

10.3 Access

- Private members worden wel geërfd maar niet rechtstreeks toegankelijk.

10.4 Constructors

- Base wordt eerst geconstrueerd.
- Als base geen default ctor heeft: base ctor expliciet callen in init-list.

10.5 Overriding vs overloading

- Overriding: zelfde signature, andere implementation in derived.
- Overloading: zelfde naam, andere parameters (kan in dezelfde class).

11) Polymorfisme (**virtual**)

11.1 Late binding

- Zonder **virtual** krijg je early binding via base pointers/references.
- Met **virtual** wordt de juiste derived method gekozen op runtime.

11.2 Virtual destructors

- Als je **delete** doet via base pointer, moet base destructor **virtual** zijn, anders blijft derived cleanup uit.

12) Abstract classes & "interfaces"

12.1 Pure virtual

```
virtual std::string getSound() const = 0;
```

12.2 Belangrijk

- Abstract class kan je niet instantiëren.
- Derived moet pure virtual methods implementeren om concreet te worden.

13) Pointers & references

13.1 Pointer basics

- Pointer = address.
- **&x** (address-of), ***p** (dereference).
- Object via pointer: **ptr->method()**.

13.2 References

- Reference = alias:

```
int &ref = x;
```

13.3 Cursuswaarschuwing rond **&**

- Links van **=**: reference.
- Rechts van **=**: address-of.

14) Dynamic memory (stack vs heap)

14.1 Stack

- Automatische storage (local vars), beperkt.

14.2 Heap

- Dynamische allocatie; programmer moet vrijgeven.

14.3 `new/delete`

```
int *p = new int;
*p = 42;
delete p;
```

14.4 `new[]/delete[]`

```
int *arr = new int[5];
// ...
delete[] arr;
```

14.5 Destructors

- Gebruik destructors om resources op te ruimen (zeker als je heap memory beheert).

15) Exceptions

15.1 Waarom

- Return codes mixen error flow met normale flow.
- Constructors kunnen geen return codes geven.

15.2 `try/catch/throw`

```
try {
    if (number < 0) throw std::string("Complex Root Exception");
} catch (const std::string &msg) {
    std::cout << msg << std::endl;
}
```

16) File streams (<fstream>)

16.1 Types

- `std::ifstream` (read), `std::ofstream` (write), `std::fstream` (read+write).

16.2 Openen + check

```
std::ifstream input;
input.open("hello.txt");
if (!input) {
    std::cerr << "Could not open" << std::endl;
}
```

16.3 Lezen

- `std::getline(input, line)` voor lijnen.

16.4 State

- `good()`, `fail()`, `bad()`, `eof()` en `clear()`.

17) Operator overloading

17.1 Beperkingen

- Arity blijft gelijk.
- Geen nieuwe operatoren.
- Niet overloadbaar: `., .*`, `::`, `?:`.

17.2 Patterns

- Free function, member function, of `friend` (voor toegang tot private + symmetrische cases).
- `<<` en `>>` voor streams zijn klassieke voorbeelden.

18) `std::vector`

18.1 Basis

- "Growable array".
- Include: `<vector>`.

18.2 Methods

- `.size()`, `.push_back()`, `.clear()`.
- `[]` (geen bounds check) vs `.at()` (wel bounds check, kan exception geven).

18.3 Sorteren

- `std::sort` uit `<algorithm>` op iterators, met optional comparator.
-

19) Snelle examen-checklist

- Compilatiepipeline + separate compilation.
- Makefile: target/deps/commands + TAB; `all/clean`.
- Headers: include guards; geen `using namespace` in headers.
- Constructors: default ctor-regel; init-list; "most vexing parse".
- Composition vs inheritance (eerst composition overwegen).
- Overloading vs overriding.
- Polymorfisme: `virtual` + virtual destructor.
- Pointers/references: `&`, `*`, `->`; arrays decay.
- Heap: `new/delete`, `new[]/delete[]`, destructors.
- Exceptions: `try/catch/throw`.
- File streams: open modes + state flags.
- Operator overloading: beperkingen + patterns.
- `std::vector`: groei, `.at()` vs `[]`, `std::sort`.